GOSIM®
GLOBAL OPEN-SOURCE INNOVATION MEETUP

GOSIM Shenzhen 2026

10月16-17日 October 16-17　　深圳·南山伊敦酒店 Aden Hotel, Shenzhen

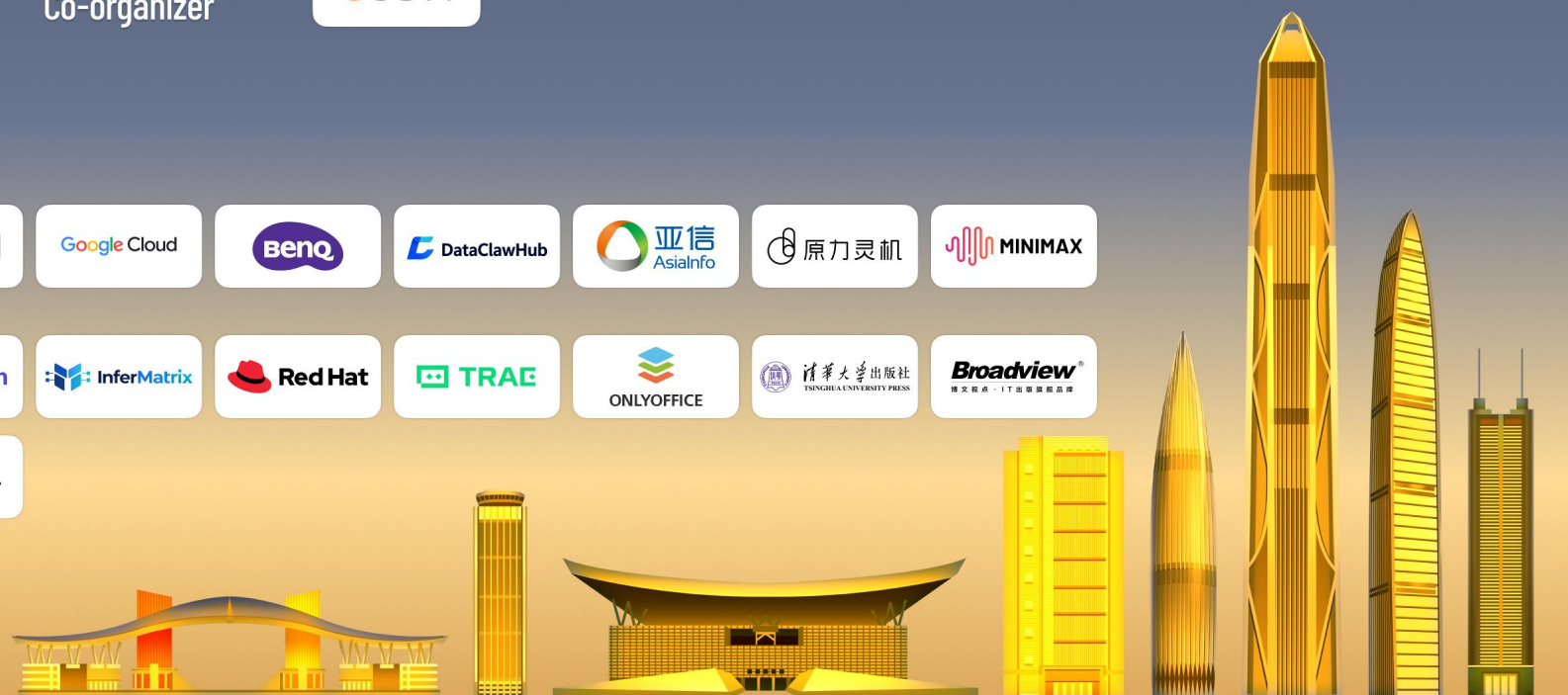
主办方 Host: GOSIM

承办方 Co-organizer: CSDN

白金赞助商 Platinum Sponsor: HUAWEI

金牌赞助商 Gold Sponsor: openJiuwen, KIMI, Google Cloud, BenQ, DataClawHub, 亚信 AsiaInfo, 原力灵机, MINIMAX

白银赞助商 Silver Sponsor: AMD, TaoToken, InferMatrix, Red Hat, TRAE, ONLYOFFICE, 清华大学出版社 TSINGHUA UNIVERSITY PRESS, Broadview 博文视点 · IT出版旗舰品牌, 清湛智造, CoHDL



GOSIM®
GLOBAL OPEN-SOURCE INNOVATION MEETUP

GOSIM Shenzhen 2026

智能体操作系统与应用

Agentic OS & App



GOSIM®
GLOBAL OPEN-SOURCE INNOVATION MEETUP

GOSIM Shenzhen 2026

Topic

Agentic Harness工程：可观测性驱动的编码智能体Harness自动进化

Agentic Harness Engineering: Observability-Driven Automatic Evolution of Coding-Agent Harnesses



林佳航 Jiahang Lin

复旦大学 FudanNLP 实验室硕士研究生

MMaster's Student, FudanNLP Lab, Fudan University

CONTENTS

Part 01 什么是 Harness

Part 02 Harness 组件分析

Part 03 Harness 优化目标

Part 04 Harness 优化方法

Part 05 展望与挑战



01

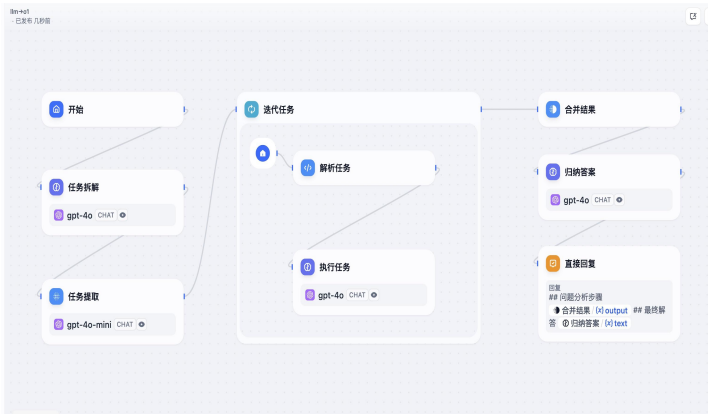
什么是 Harness



背景：大模型应用形态

大模型 | 工作流智能体 | 能动性智能体







承上启下
在互联网基础上引入一组新的能力

进入能动性阶段：
意图理解，目标遵循，
长程规划，工具交互

Vibe Code
重塑软件和其它行业

ChatGPT 发布 2022.11.30

Claude Code 发布 2025.2.24

Karpathy预言 2025.12.27

相变开始：Boris Cherny -零代码 2025.12.27

Cowork 发布 2026.1.12

红杉 宣布AGI到来 2026.1.15

“龙虾” 发布 2026.1.25

“龙虾” 社区 2026.1.30

SaaS “剃头” 安全软件 “剃头” 2026.2.3&2.20

Claude Code 一周年 2026.2.24

Anthropic 2月新增$60亿美金 2026.2.28

ChatGPT 5.4 2026.3.20

Sora关停 OpenAI转型

背景：从工作流到能动性智能体

工作流智能体模式

依赖人工经验构建	工作流数量多时 难以选择最优方案	任务边界被设计时固定 缺乏灵活性	结构化、稳定、易复现	效果可追溯 问题可定位

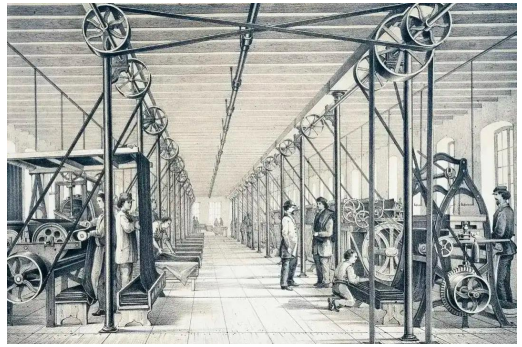
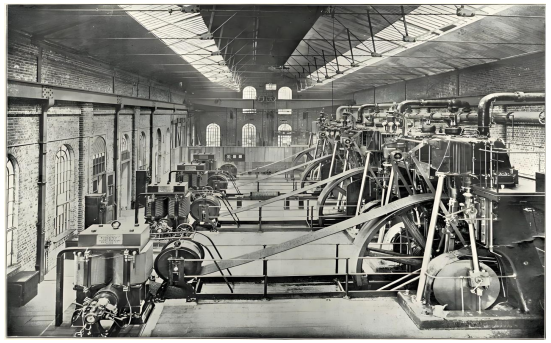
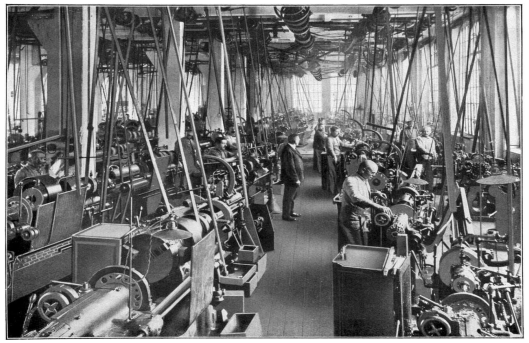
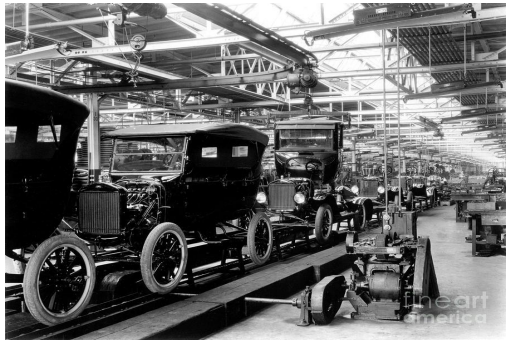
传统工作流智能体是“人工定义能动性”的产物

下一阶段 迈向自我编排与动态进化的能动系统

主动决策	环境交互	自适应调用	场景广泛
允许 LLM 直接执行动作 并根据环境反馈决定下一步操作	环境本身也可调用其他 LLM 实现多层智能体协同	传统的多种 Agentic Workflow（路由、并行、协调器等）都可被智能体按需、自适应地动态组合	适合解决步骤数量不确定、流程无法预设的开放性任务

背景：能动性的发挥依赖驾驶舱

历史类比：电力的电气化改造

初始形态	1880-1890	1890-1907	1910-1930
			
巨大的蒸汽机是通过皮带传输动能到工厂的各个地方	仍然还是蒸汽机提供动力，通过一个小的发电机来把照明输送到工厂的各个位置	沿用蒸汽机体系，仍受其限制，只因电机马力更大，皮带排布更密	电气化改造催生新工程体系，马达可在任意位置提供动能，工厂摆脱动力轴限制，使福特得以构建流水线工厂。

能动性的真正潜能，往往要经历 驾驶舱迭代+工程体系重构 才能释放

如同电气化重塑工厂，大模型的独特能动性要改变社会，必须同步完成驾驶舱设计与工程体系建设

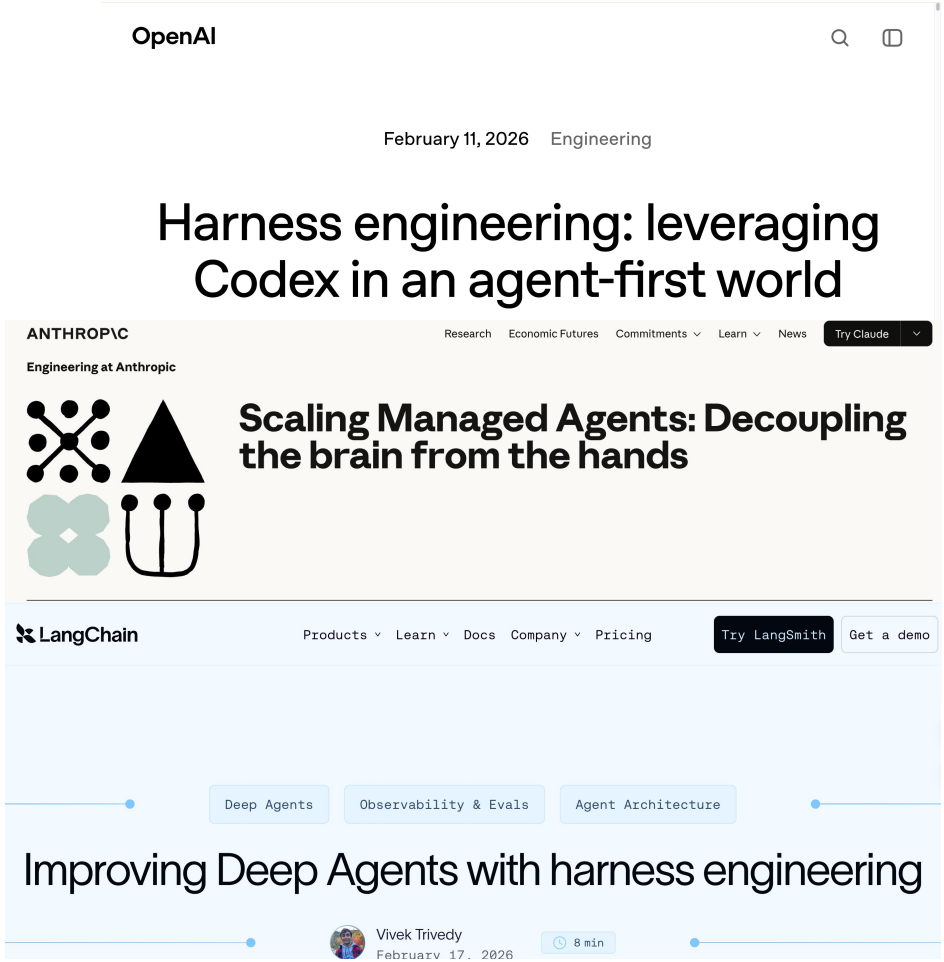
Harness 定义



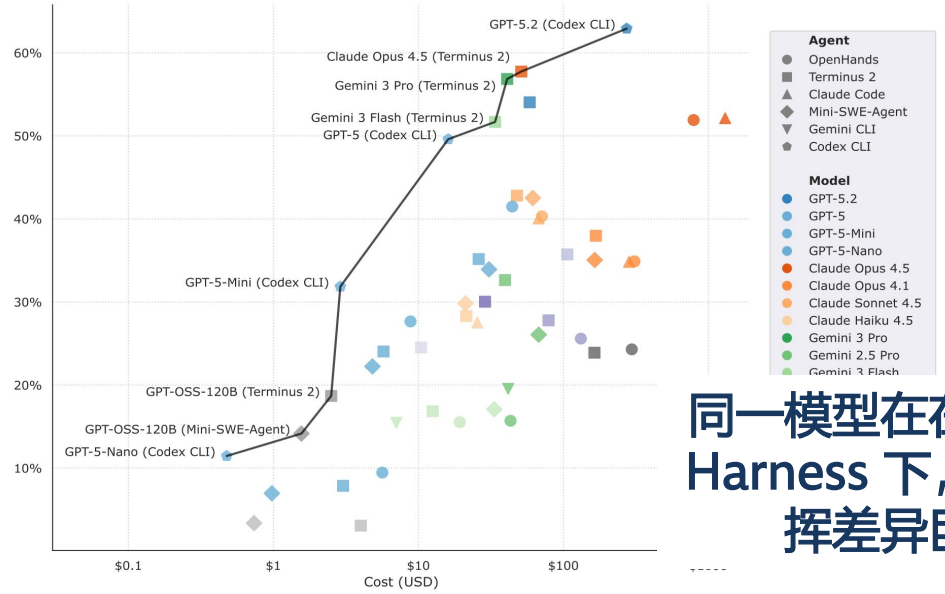
牛津词典释义

1. to put a harness on a horse or other animal; to attach a horse or other animal to something with a harness
2. harness something to control and use the force or strength of something to produce power or to achieve something
   利用某物的力量或强度来控制并产生动力或达成某种目的

Harness 重要性



前沿实验室研究热点



同一模型在在不同的 Harness 下，性能发挥差异巨大

Figure 5: The Pareto frontier of agent performance showing the tradeoff between performance and cost (log scale) on Terminal-Bench 2.0.



工业界招聘热点

02

Harness 组件分析

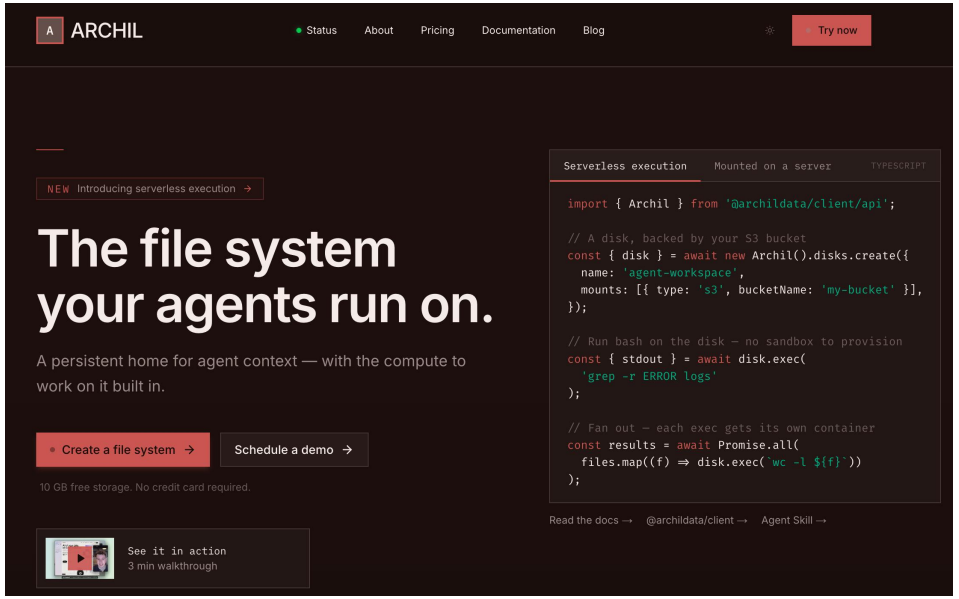
本体论视角：信息层、执行层、反馈层

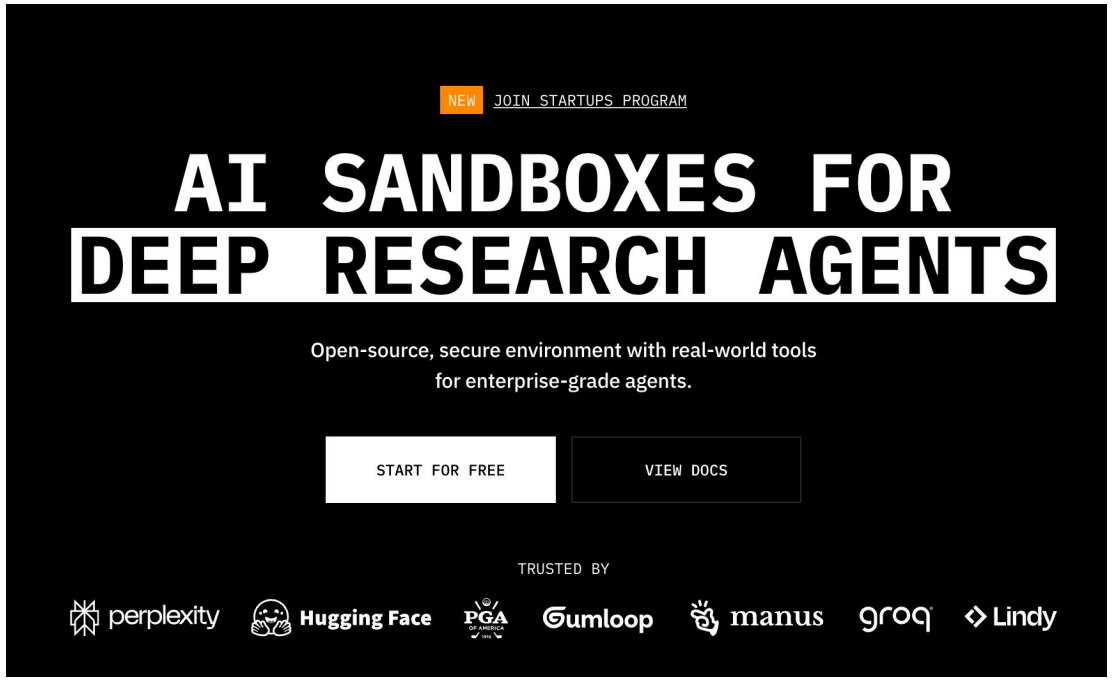


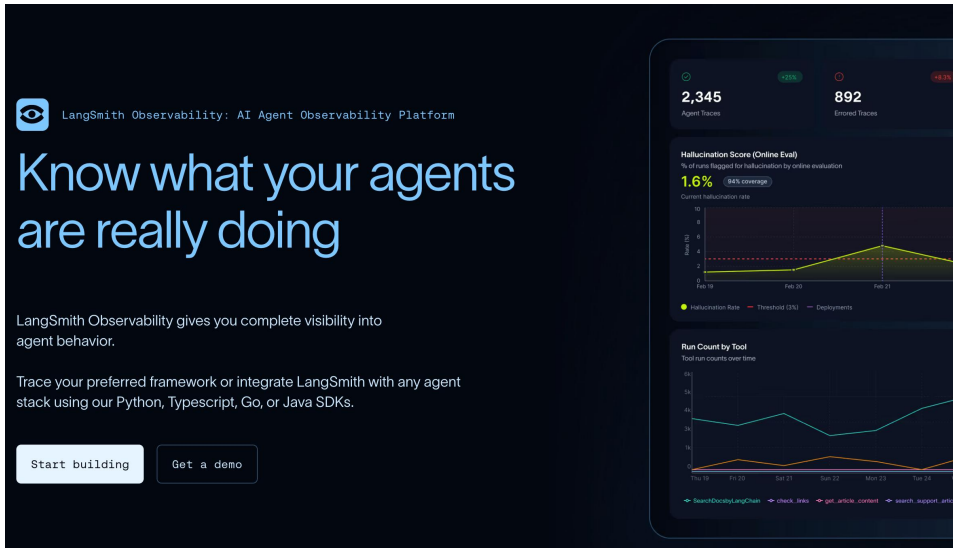
信息层： 提示词、记忆、Skill、会话管理

执行层： 上下文压缩、智能体编排、调度、沙盒

反馈层： 轨迹记录、评测







信息层组件：技能（Skill）

定义

可被 agent 按需调用的能力单元

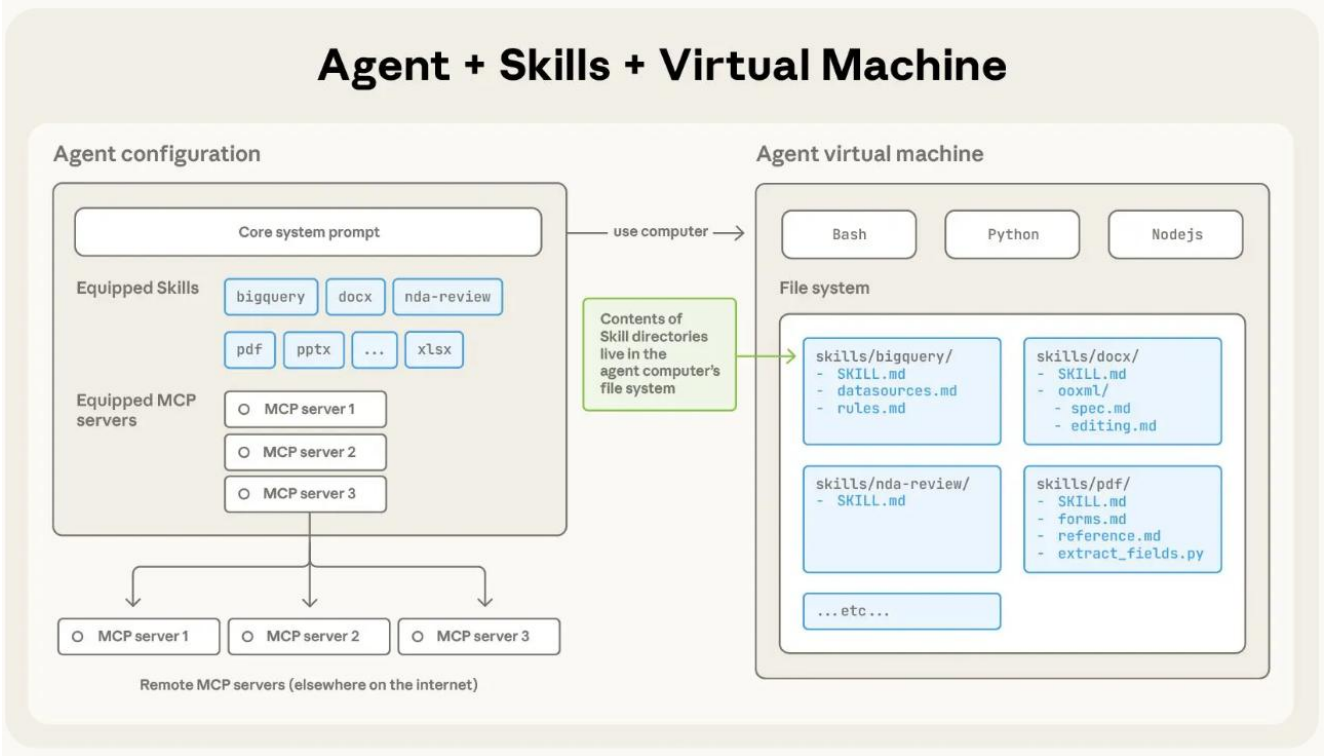
- 轻量级索引+ 完整实现

机制

- 渐进式披露：只有轻量级索引进系统提示词，完整实现按需加载
- 触发方式：显式命令调用或模型自主匹配
- 可组合：技能之间互相引用，形成能力图谱

解决痛点

- 上下文稀缺性，跨会话泛用性，能力可沉淀性



https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills

执行层组件：许可（Permission）

定义

工具调用的授权

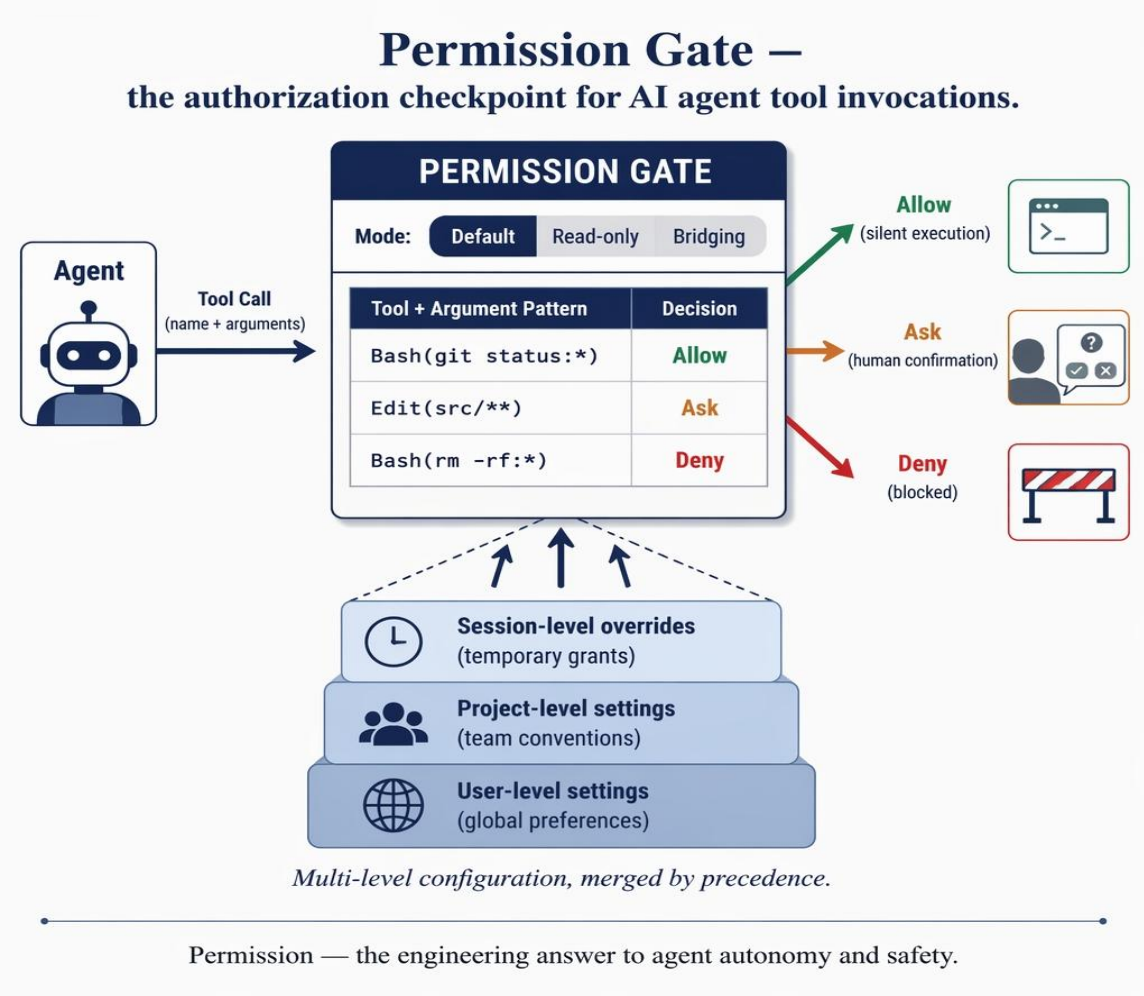
- 决定哪些动作可静默执行、哪些必须人类确认、哪些直接禁止

机制

- 三态分类：同意 / 询问 / 拒绝，按工具 + 参数模式匹配实现精确控制
- 多级配置：用户级（全局习惯）→ 项目级（团队约定）→ 会话级（临时放行）
- 模式切换：默认 / 只读 / 桥接
- 审查时机：在工具调用前执行许可检查

解决痛点

- 安全与效率平衡，权限边界的精准控制



执行层组件：工具 （Tool）

工具是智能体接触外部世界的接口，**按调用模式分两类**

同步工具

调用模型

- 调用 → 等结果 → 输出整段进入上下文

典型例子

- 文件读写工具、网络搜索工具

适用场景

- 短任务、结果立即可用、需要立即进入推理上下文

局限

- 长程任务会占满上下文、运行时间线性增加

异步工具

调用模型

- 后台启动 → 拿到任务句柄 → 按需拉取输出

典型例子

- 命令行工具、子agent调用、定时任务工具

三大收益

- 上下文减压 • 并行执行 • 可恢复可调度

意义

- 大幅提高agent执行效率，避免 agent 上下文被单次长输出淹没—— 长程自主运行的基础

异步化是 agent 从对话切换到长程进程的工程化答案

反馈层组件：轨迹数据库（SmithDB）

定义

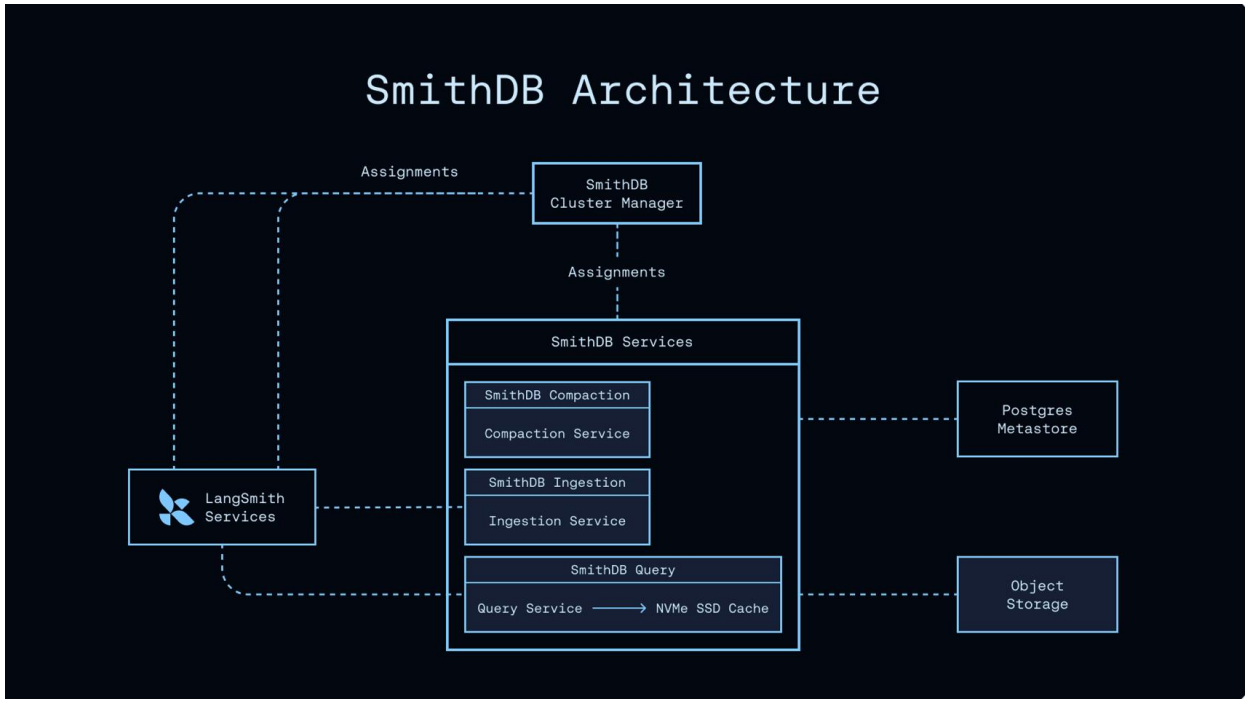
SmithDB —— 为智能体轨迹观测专门构建的分布式数据库（传统数据库不适配：嵌套深、调用段持久数小时、多查询模式叠加）

实现

- **无状态服务层**：写入 / 查询 / 压实合并 解耦伸缩
- **对象存储**：持久化轨迹本体，无本地磁盘管理
- **元数据库**：基于 PostgreSQL，存分段元数据与索引

优势

- 核心查询延迟 71 – 400 ms，比原方案快12倍
- 对象存储底座，多云 / 自托管友好



https://www.langchain.com/blog/introducing-smithdb

03

Harness 优化目标

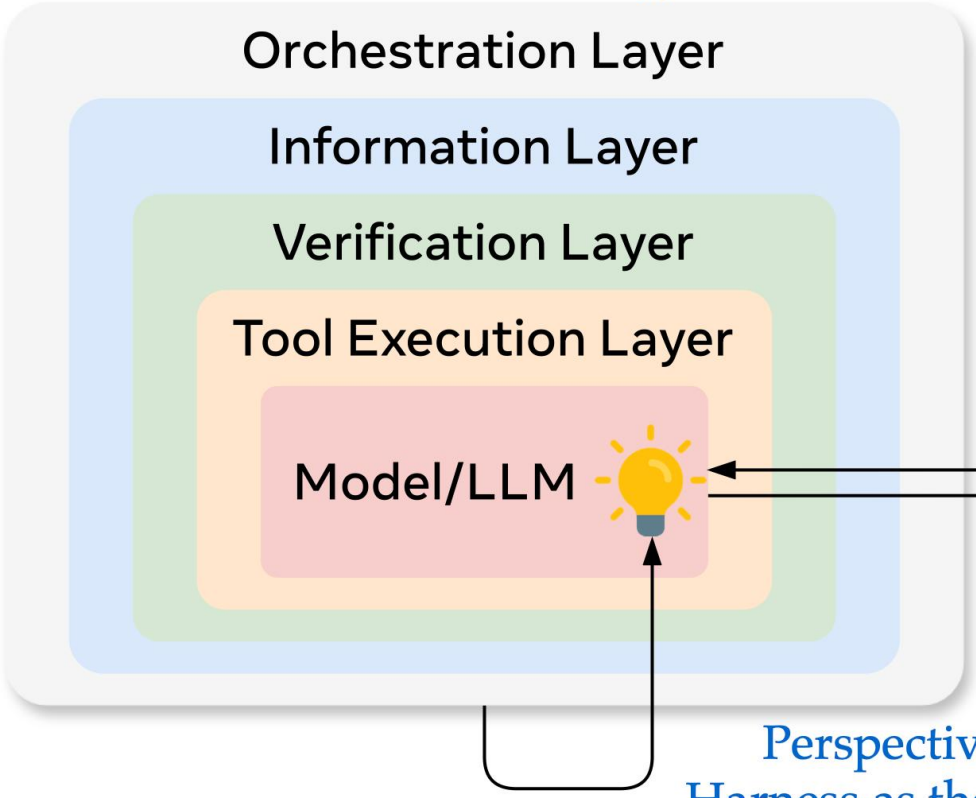
上下文管理与行为控制



Harness 优化方向

视角 1：Harness 作为包装

Perspective 1:
Harness as the *wrapper*



Harness 通过**控制**模型和外界的双向**上下文流**，来影响模型的行为

视角 2：Harness 作为代理

Perspective 2:
Harness as the *proxy*



- Harness 作为环境的代理
- Harness 作为用户的代理

Perspective 3:
Harness as the *dataset*

视角 3：Harness 作为数据集

AutoHarness: improving LLM agents by automatically synthesizing a code harness

Xinghua Lou, Miguel Lázaro-Gredilla, Antoine Dedieu,
Carter Wendelken, Wolfgang Lehrach, Kevin P. Murphy
Google DeepMind
{xinghua,lazarogredilla,adedieu,cwendelken,wpl,kpmurphy}@deepmind.com

March 5, 2026

Abstract

Despite significant strides in language models in the last few years, when used as agents, such models often try to perform actions that are not just suboptimal for a given state, but are strictly prohibited by the external environment. For example, in the recent Kaggle GameArena chess competition, 78% of Gemini-2.5-Flash losses were attributed to illegal moves. Often people manually write "harnesses" around LLMs to prevent such failures. In this paper, we demonstrate that Gemini-2.5-Flash can automatically synthesize such a code harness, using a small number of rounds of iterative code refinement given feedback from the (game) environment. The resulting harness prevents all illegal moves in 145 different TextArena games (both 1-player and 2-player), enabling the smaller Gemini-2.5-Flash model to outperform larger models, such as Gemini-2.5-Pro. Pushing our technique to the limit, we can get Gemini-2.5-Flash to generate the entire policy in code, thus eliminating the need to use the LLM at decision making time. The resulting code-policy receives a higher average reward than Gemini-2.5-Pro and GPT-5.2-High on 16 TextArena 1-player games. Our results show that using a smaller model to synthesize a custom code harness (or entire policy) can outperform a much larger model, while also being more cost effective.

Google DeepMind，2603.03329

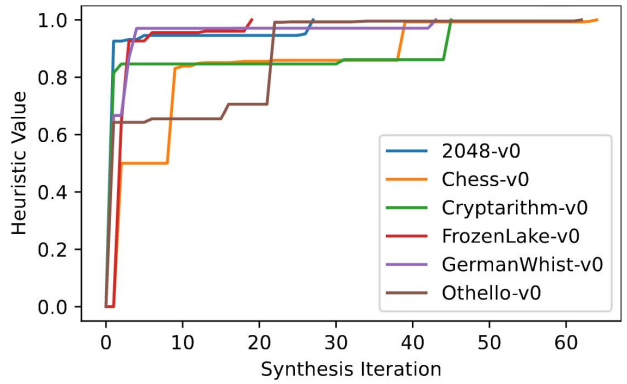


Figure 2: Fraction of legal moves vs number of code refinements for a selection of 6 games.

4.2 Evaluation

We now turn to evaluating performance of agents during actual game play. For reasons of efficiency, we focus our results on 16 1P games and 16 2P games, rather than using all 145 games. We evaluate the following agents: Gemini-2.5-Flash, Gemini-2.5-Pro and Gemini-2.5-Flash+Harness (ours).[2] We use the same optimized prompt in all experiments. For 1P games, we run 20 matches and use the reward as the evaluation metric. For 2P games, we run 40 matches with random seeds, split evenly between our method being the first or second player, and we use the average win/draw/loss rate as the evaluation metrics.

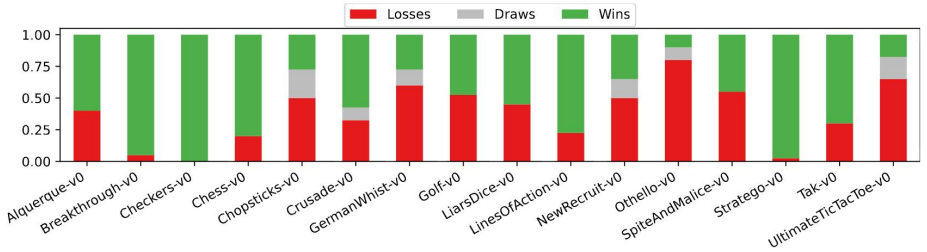


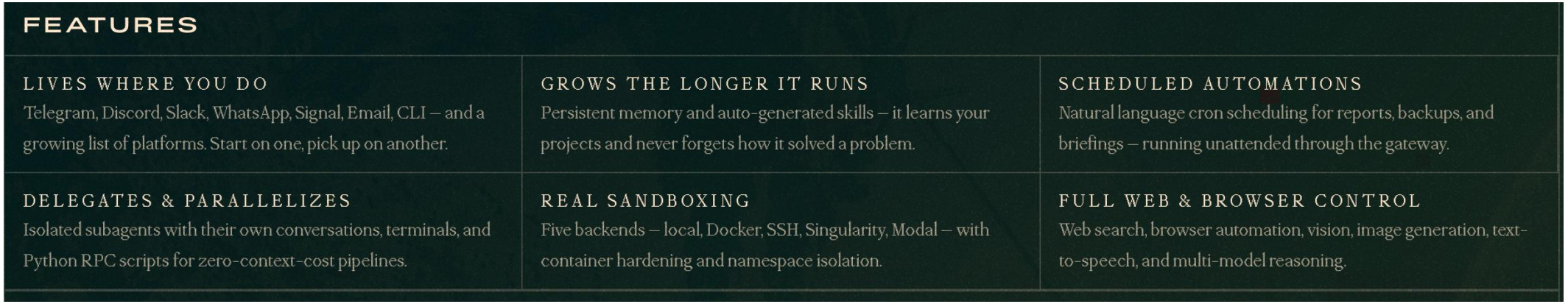
Figure 3: Win/lose/draw rate of our method vs Gemini-2.5-Pro for each of the 16 2P games.

效果：Harness 作为环境的代理，
用 hook 来约束智能体行为，提升智能体执行动作的正确性

Harness 优化方向：用户偏好



效果：Harness 作为用户的代理，
沉淀用户偏好，优化技能组件，让用户不必一次次重复输入



Hermes Agent 功能：与用户共同成长，实现长期进化

https://hermes-agent.nousresearch.com/

04

Harness 优化方法

从人工优化到自动优化 · 从独立优化到联合优化



优化空间

模型-Harness-环境

一般流程

1. 评测：在一个场景中运行 Harness
2. 分析：从运行轨迹中分析错误模式
3. 改进：修改对应的 Harness 组件

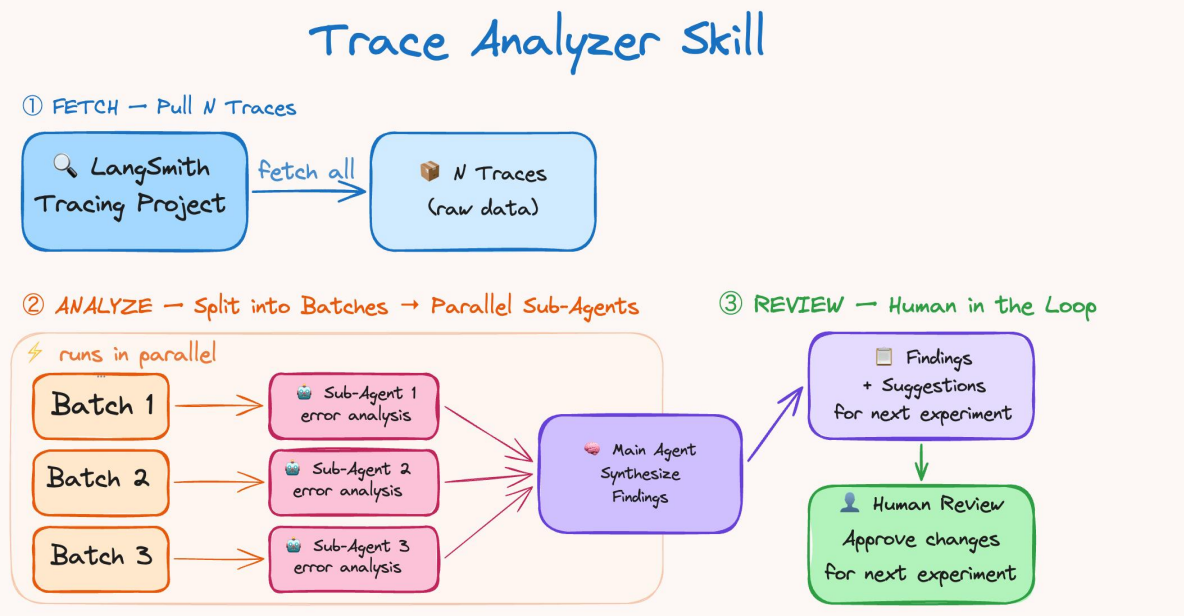
发展方向

评测：长时程、多工具任务
分析：从人工分析到自动分析
改进：从独立改进到联合改进

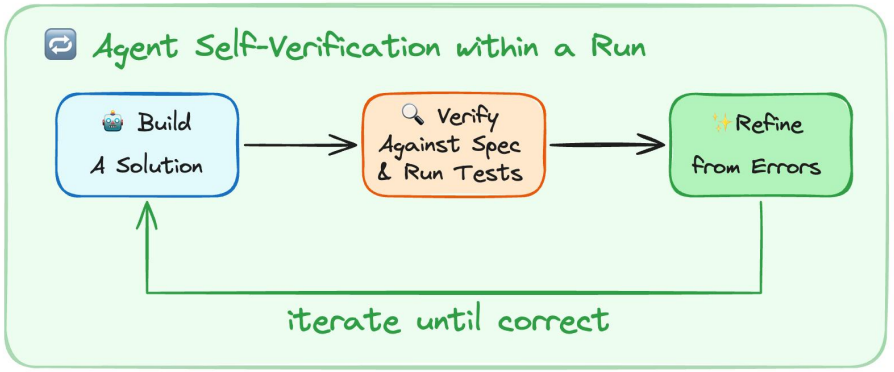


人工优化：轨迹分析

瓶颈：轨迹长达50k, 人类阅读时间过长



Run	What Changed	Score
Baseline	Baseline prompt for coding, file system tools, planning, etc.	52.8%
+ Custom Prompt & Middleware	Focus on a Build-Verify Loop, Inject environment context. Add loop protection, and timeout warnings	63.6%
+ Adaptive reasoning	xhigh+high reasoning	66.5%



方案：从纯人工分析转入半Agent化，利用子智能体对轨迹进行并行分析

效果：手动设计 Harness 组件，性能提升 13.7%

Improving Deep Agents with harness engineering

单组件自动优化：Skill

论文：Trace2Skill

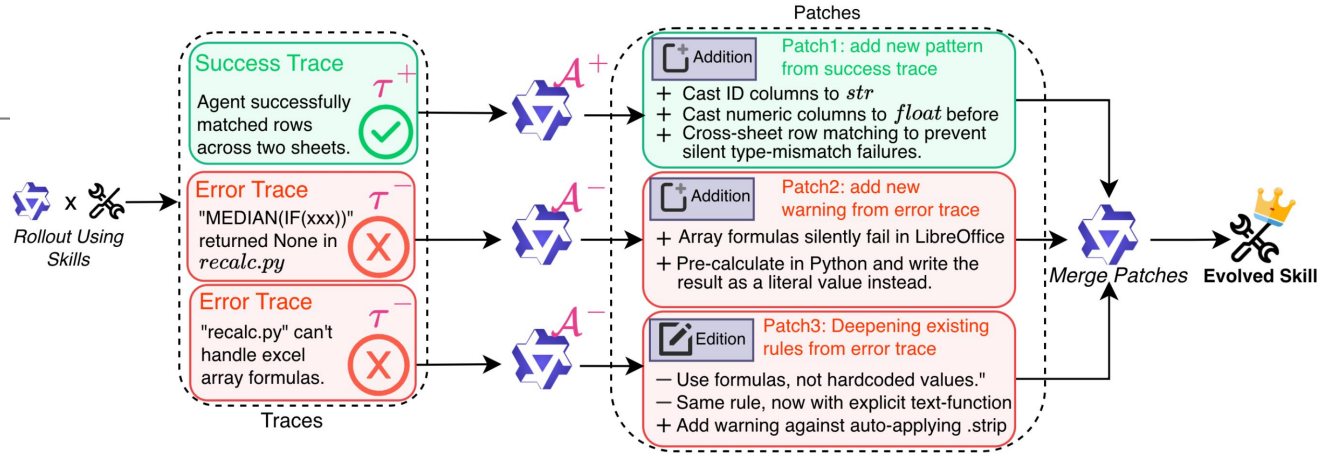
Alibaba Qwen Applications Business Group

Trace2Skill: Distill Trajectory-Local Lessons into Transferable Agent Skills

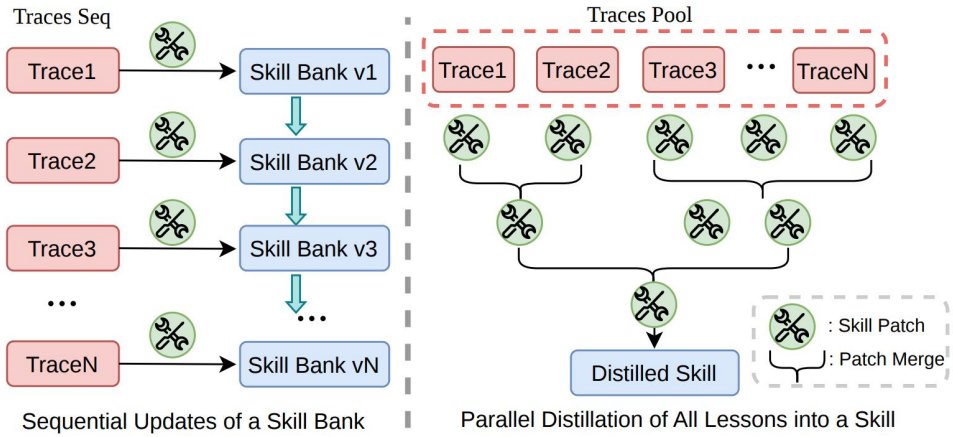
Jingwei Ni[§,*2,3], Yihao Liu[§,*4], Xinpeng Liu[§,*4], Yutao Sun[§,*5], Mengyu Zhou[†1], Pengyu Cheng[1], Dexin Wang[1], Erchao Zhao[1], Xiaoxi Jiang[1] and Guanjun Jiang[1]

[1]Qwen Large Model Application Team, Alibaba, [2]ETH Zürich, [3]University of Zurich, [4]Peking University, [5]Zhejiang University

[*]Work done during an internship at Alibaba. [†]Corresponding author. [§]Core Contributors.



Trace2Skill 框架通过并行分析LLM代理的执行轨迹，并归纳整合经验教训，**自动创建**和优化可迁移的、领域特定技能。



联合优化：多组件端到端自动优化

Meta-Harness 首个提出端到端优化Harness的方法

- 效果：Agent迭代出的Harness, 超越手工优化的 Harness
- 核心：保存所有历史，供agent观测

Meta-Harness: End-to-End Optimization of Model Harnesses

Yoonho Lee Stanford
Roshen Nair Stanford
Qizheng Zhang Stanford
Kangwook Lee KRAFTON
Omar Khattab MIT
Chelsea Finn Stanford

Project page w/ interactive demo: https://yoonholee.com/meta-harness/
Optimized harness: https://github.com/stanford-iris-lab/meta-harness-tbench2-artifact

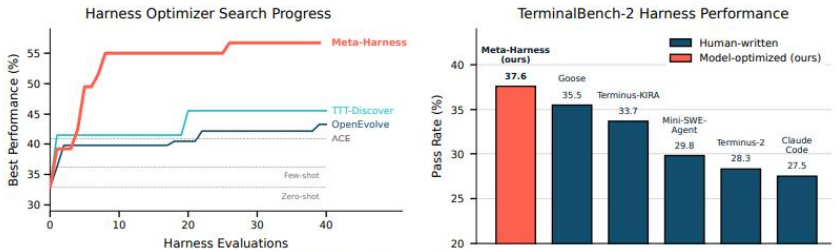


Figure 1: **(Left)** On text classification, Meta-Harness outperforms the best prior hand-designed harnesses (ACE) and existing text optimizers (TTT-Discover, OpenEvolve), matching the next-best method's final accuracy after just 4 evaluations. **(Right)** On TerminalBench-2, Meta-Harness outperforms all reported `Claude Haiku 4.5` harnesses.

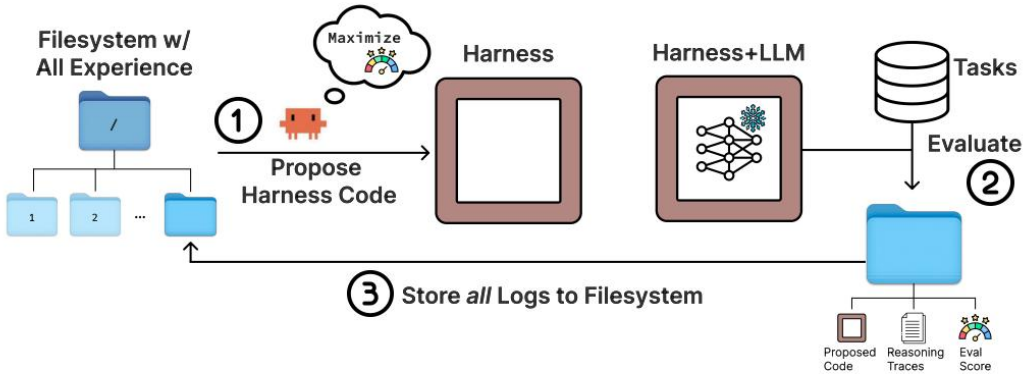


Figure 2: **Meta-Harness search loop. (1)** An agent reads a filesystem containing all prior candidates' source code, execution traces, and scores, and proposes a new harness. **(2)** We evaluate the proposed harness on evaluation tasks. **(3)** All logs (proposed code, reasoning traces, evaluation scores) are stored in the filesystem in a new directory, and the loop repeats.

Method	History	Log content	MTok/iter
OPRO [51]	Window	past (solution, score) pairs	0.002
TextGrad [53]	Last	textual feedback on current artifact	0.015
AlphaEvolve [35]	Window	program database + eval. scores	0.022
GEPA [1]	Summary	reflective feedback from rollout traces	0.008
Feedback Descent [26]	Summary	comparison + textual feedback	0.012
TTT-Discover [54]	Window	prev. solution fragment	0.026
`Meta-Harness`	**Full**	***all*** logs and scores	10.0

Table 1: **Comparison of text optimization methods and their settings.** Each row represents a method collapsed across tasks. `Mtok/iter` is our best estimate of the full context generated from one evaluation of a text artifact in the *largest setting considered in each paper*. This paper considers settings that yield orders-of-magnitude more context per artifact evaluation.

https://arxiv.org/abs/2603.28052

Agentic Harness Engineering：
可观测性驱动的代码智能体 Harness 自动进化



Github 收获 900+收藏

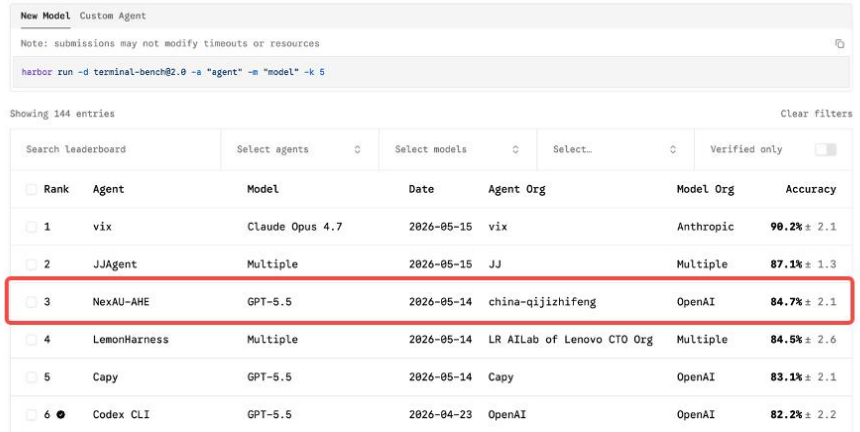


X 上获得 10 万+浏览



头部媒体报道

terminal-bench@2.0 Leaderboard



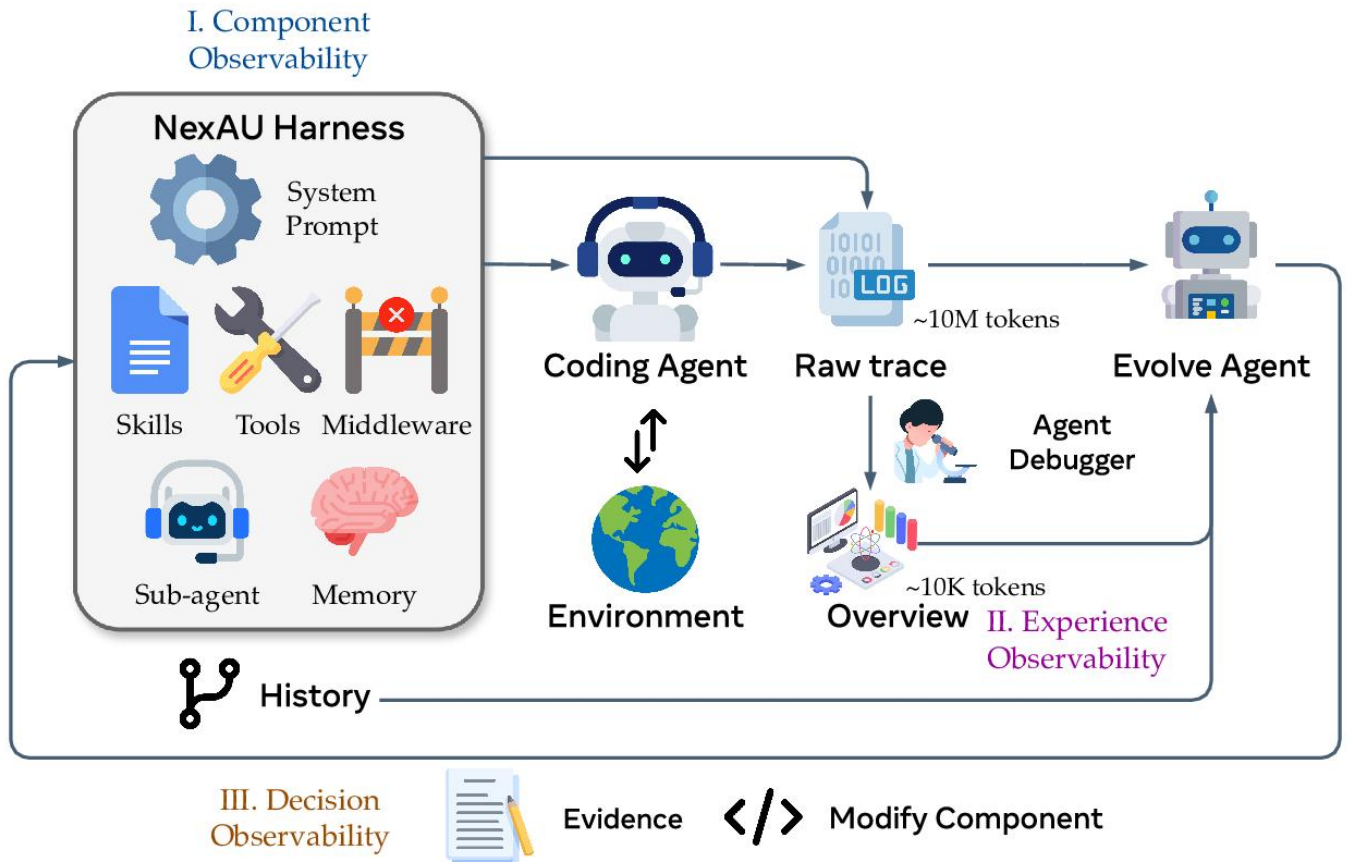
权威榜单全球前三

核心：搭建了一套完善的**观测平台**，对全阶段做详细而精确的记录。分三部分：

1. NexAU 提供各部分解耦的 Harness，提供 Harness 组件**的可观测性**；

2. Agent Debugger 实现原始轨迹的分层溯源，实现**反馈结果的可观测性**；

3. Evolve Agent 基于 git 溯源的组件历史、反馈结果，构建证据驱动的完整修改链路，对相应组件进行修改，实现**优化行为的可观测性。**

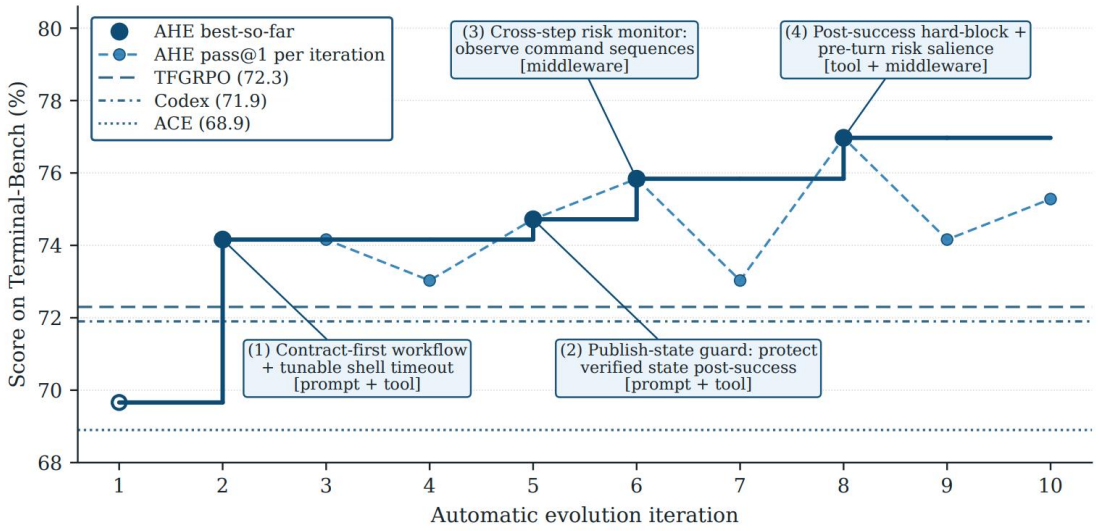


Evolve agent改动举例：

1. **Tools:** 给bash工具增加超时、后台模式

2. **系统提示**：增加长期规则，例如：完成前先按照标准评测自检

3. **中间件**：循环检测、错误检测





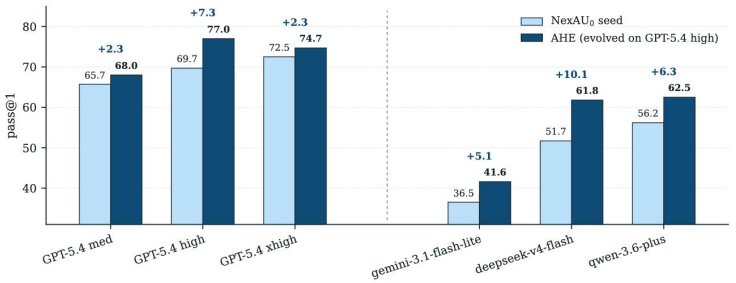


Figure 3: Cross-model transfer on Terminal-Bench 2, 89 tasks. The AHE workspace evolved on GPT-5.4 high is re-evaluated on each base without further evolution, paired against the NexAU$_0$ seed on the same base. All bases gain over the seed, with cross-family bases gaining more than within-family ones.

优化得到的 Harness 跨模型迁移性良好

Table 5: Cost-efficiency on SWE-bench-verified, reported as Succ/Mtok, the expected successes per million tokens. Values are derived from Table 2 as pass@1 $\times$ 10^3/Tokens k. Higher is better. Per-row bold marks the best.

Repo	N	ACE	TF-GRPO	NexAU$_0$	**AHE**
All	500	1.10	1.27	1.43	**1.64**
django	231	1.12	1.35	1.50	**1.67**
sympy	75	1.15	1.19	1.43	**1.48**
sphinx-doc	44	0.62	0.78	0.93	**1.07**
matplotlib	34	1.14	1.33	1.51	**1.88**
scikit-learn	32	2.08	2.48	3.06	**3.40**
pydata	22	1.37	1.50	2.00	**2.15**
astropy	22	1.08	1.26	0.82	**1.81**

Harness 跨数据集迁移性良好

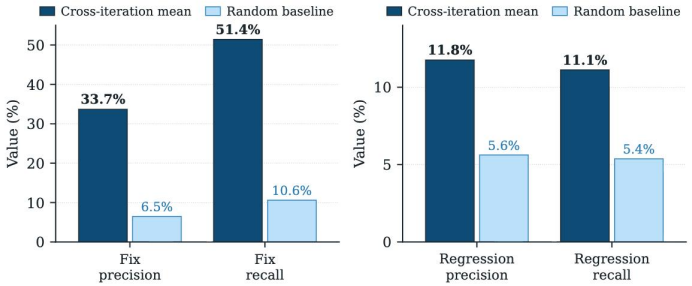


Figure 4: Cross-iteration mean precision and recall of the evolve model's self-predictions across 9 evaluation rounds of the GPT-5.4 AHE loop on Terminal-Bench 2, alongside the random-prediction baseline. Left: fix predictions. Right: regression predictions.

分析实验：预测优化准确性

Table 3: Component-level ablations on Terminal-Bench 2. Each "+X only" row swaps a single AHE component into the NexAU$_0$ seed; per-column best is bolded. Most single-component swaps already beat the seed, and the single-component gains overshoot full AHE's aggregate, indicating components interact non-additively rather than stacking cleanly.

Variant	All 89 tasks	Easy 4 tasks	Medium 55 tasks	Hard 30 tasks
NexAU$_0$	69.7%	87.5%	78.2%	51.7%
+ memory only	75.3%	50.0%	83.6%	**63.3%**
+ tool only	73.0%	75.0%	87.3%	46.7%
+ middleware only	71.9%	**100.0%**	81.8%	50.0%
+ system_prompt only	67.4%	75.0%	78.2%	46.7%
AHE full	**77.0%**	**100.0%**	**88.2%**	53.3%

消融实验：各组件的价值沉淀

GOSIM

GOSIM Shenzhen 2026

05

展望与挑战

Harness 与模型协同进化



模型–Harness 协同进化

- **长期目标**：模型-Harness co-evolve（协同进化）
- Cat Wu（Claude Code 产品经理）：“**模型会把你的 harness 当早餐吃掉。**”
- Philipp Schmid（Google DeepMind）：“The Harness is the Dataset”
  由 Harness 运行时产生的轨迹是下一代模型训练的基础语料
- 从*在手状态*（Vorhandenheit）变成*上手状态*（Zuhandenheit）

What Comes Next?

We are heading toward a convergence of training and inference environments. We see a new bottleneck being context durability. The Harness will become the primary tool for solving "model drift". Labs will use the harness to detect exactly when a model stops following instructions or reasoning correctly after the 100th step. This data will be fed directly back into training to create models that don't get "tired" during long tasks.

As builders and developers the focus should shift:

1. **Start Simple:** Do not build massive control flows. Provide robust atomic tools. Let the model make the plan. Implement guardrails, retries and verifications.
2. **Build to Delete:** Make your architecture modular. New models will replace your logic. You must be ready to rip out code.
3. **The Harness is the Dataset:** Competitive advantage is no longer the prompt. It is the trajectories your Harness captures. Every time your agent fails to follow an instruction late in a workflow can be ued for training the next iteration.

https://www.philschmid.de/agent-harness-2026

- Harness 是 2026 年重要方向
  - Harness 保证了智能体在运行长期任务的稳定性、鲁棒性
- Harness 包括信息层、执行层、反馈层
  - Harness 需要系统性开发
- Harness 的优化方向是行为可控性和上下文传递通畅
  - 双向的上下文流动
- Harness 优化从人工迈向自动，从独立变成联合
  - 需要做好上下文的忠实记录和分层提取



GOSIM

GOSIM Shenzhen 2026

Thanks!



GOSIM®
GLOBAL OPEN-SOURCE INNOVATION MEETUP

GOSIM Shenzhen 2026

智能体操作系统与应用

Agentic OS & App

